

Logging Metrics and Artifacts: The Foundation of Reproducible ML

A practical guide for ML engineers and data scientists implementing experiment tracking



Why Logging Matters



Lost Results

Without proper logging, valuable experimental outputs and metrics disappear forever



Performance Tracking

Metrics capture performance trends across training runs and model iterations



Reproducibility

Artifact preservation ensures experiments can be reproduced exactly

Logging creates the essential foundation for **scientific reproducibility** in AI/ML development

What Are Metrics?

Numerical values recorded during training that help evaluate **model quality & performance tradeoffs**

Quality Metrics

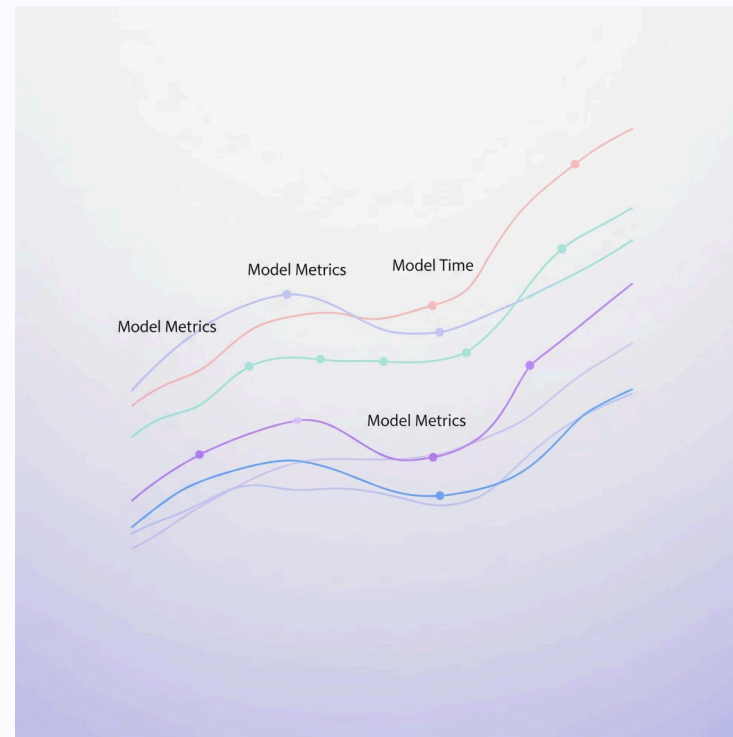
Accuracy, F1, Precision, Recall, AUC

Training Metrics

Loss, Perplexity, Gradient norms

Performance Metrics

Latency, Throughput, Memory usage



Metrics logged across epochs or steps

What Are Artifacts?

Files generated during experiments that provide **context & reproducibility**

Model Artifacts

Trained weights, checkpoints, serialized models

Visualization Artifacts

Loss curves, confusion matrices, embedding projections

Data Artifacts

Preprocessed datasets, feature statistics, sample predictions



MLflow – Logging Metrics

```
import mlflow
```

```
with mlflow.start_run():
```

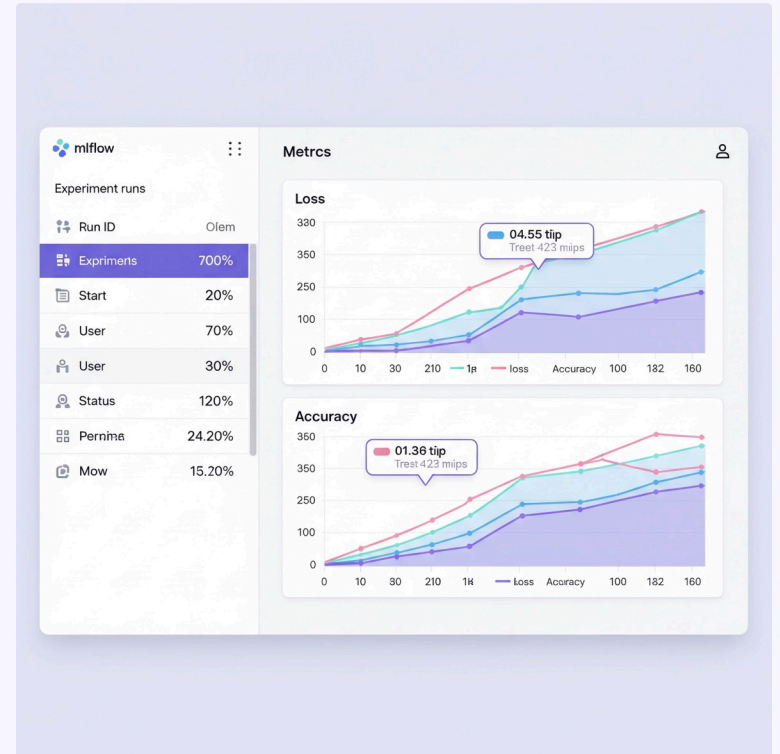
```
    mlflow.log_param("lr", 0.01)
```

```
    mlflow.log_metric("loss", 0.345)
```

```
    mlflow.log_metric("accuracy", 0.89)
```

Benefits:

- Tracks metrics for each experiment run
- View results in intuitive MLflow UI
- Supports multiple metrics per run
- Automatic experiment organization



MLflow UI displaying experiment metrics

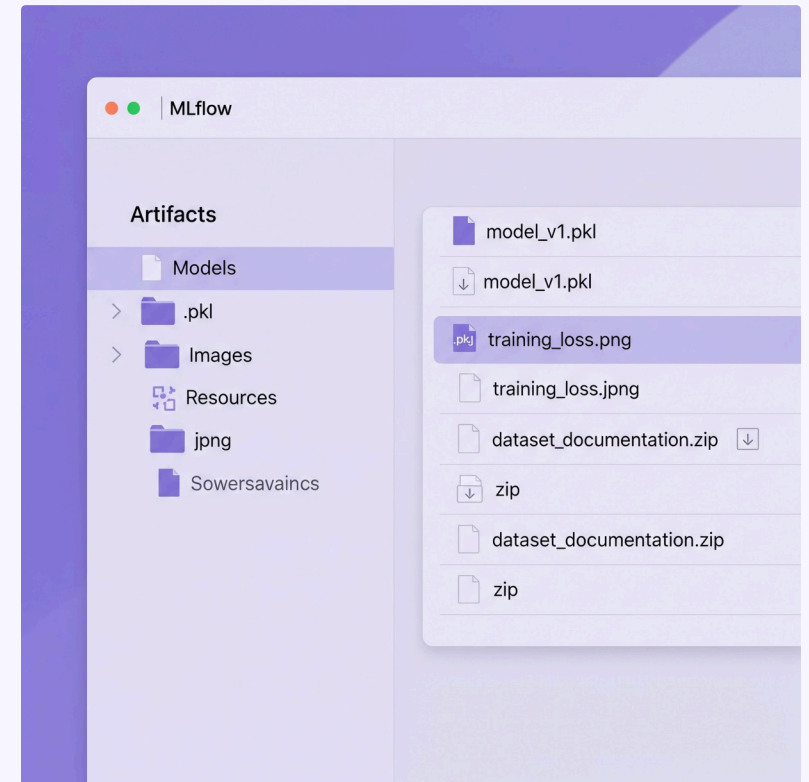
MLflow – Logging Artifacts

```
import mlflow

with mlflow.start_run():
    mlflow.log_artifact("confusion_matrix.png")
    mlflow.log_artifact("model.pth")
```

With MLflow, artifacts are:

- Stored alongside run history
- Downloadable for later comparison
- Organized by experiment ID
- Integrable with cloud storage (S3, GCS, Azure)



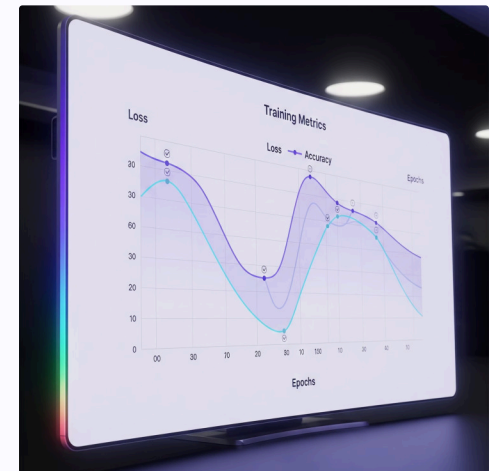
Artifacts panel in the MLflow UI

Example – Logging During Training

```
for epoch in range(5):  
    # Train one epoch  
    loss, acc = train_one_epoch()  
  
    # Log metrics at current step  
    mlflow.log_metric("loss", loss, step=epoch)  
    mlflow.log_metric("accuracy", acc, step=epoch)  
  
    # Log checkpoint every 2 epochs  
    if epoch % 2 == 0:  
        torch.save(model.state_dict(), f"model_e{epoch}.pt")  
        mlflow.log_artifact(f"model_e{epoch}.pt")
```

Key Points:

- Records metrics **per epoch/step**
- Enables plotting trends in MLflow UI
- Creates checkpoints for model versioning
- Excellent for hyperparameter comparisons



Benefits of Logging

1

Performance Tracking

Monitor model improvements over time with historical metric comparisons

2

Experiment Comparison

Easily compare multiple runs with different hyperparameters side-by-side

3

Training Stability

Debug issues like loss divergence by analyzing training curves

4

Model Preservation

Automatically save and catalog the best-performing model versions

Comprehensive logging supports **collaborative research, compliance requirements**, and accelerates development

Logging Beyond MLflow

TensorBoard

Native for TensorFlow

- Real-time visualizations
- Built-in model graph visualization
- Free and open-source

Weights & Biases

Collaborative tracking

- Team-oriented features
- Advanced report generation
- Media logging (images, video)

Comet.ml, Neptune.ai

Cloud-based platforms

- Extensive integrations
- Experiment comparison
- Artifact versioning

MLflow remains the most flexible, self-hosted option with broad framework compatibility



Common Mistakes to Avoid

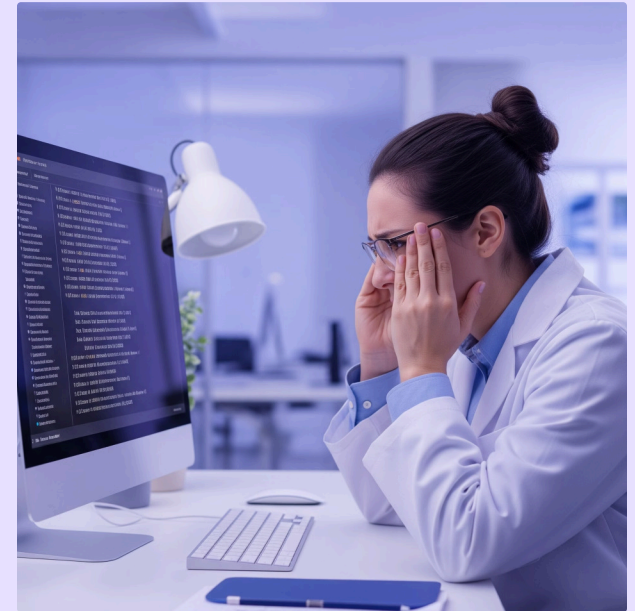
Insufficient Metrics: Logging too few metrics leads to incomplete analysis and hidden issues

Missing Checkpoints: Forgetting to save model checkpoints makes reproducing results impossible

Unversioned Storage: Storing artifacts outside version control creates confusion

Log Overwriting: Not versioning runs properly causes data loss

Poor Tagging: Failing to tag runs with hyperparameters makes comparison difficult



"I'm sure I had a model that performed better than this last week, but I can't find the configuration anywhere!"

— Every ML engineer at some point

Key Takeaways

1

Metrics Are Your Compass

Numerical performance indicators enable trends and comparisons

2

Artifacts Ensure Reproducibility

Models, plots, and datasets preserve the full experiment context

3

MLflow Simplifies Logging

Simple API with `log_metric()` and `log_artifact()` handles most needs

4

Logging Is Foundational

Without logging, AI experiments become a frustrating black box

Invest in logging
now to save
countless hours
later

